Imperial College London

MSc Artificial Intelligence, Individual Project

Closing the Loop Between Synthetic Agent Testing and Production Reality

A Production-Feedback System for Predictive Agent Reliability Testing, Validated on a Live Deployed Call-Centre Agent

Background Report

Eduardo Fernandez Salazar

2026

Table of Contents

Chapter 1 Introduction

1.1 The Deployment Gap

Organisations are deploying tool-using conversational agents into production faster than they can assure that those agents behave correctly. An agent that performs well in a developer's playground can, once it meets real users, hallucinate, call the wrong tool, or fail silently in ways that never surfaced during development. The consequences are not abstract. In a live commercial call-centre operation, an agent that quotes a customer the wrong price exposes the client brand to legal and financial liability, and a single visible failure of this kind erodes user trust faster than any accuracy metric can rebuild it. The central difficulty is therefore not whether an agent can be made to work, but whether it can be assured to work before it is exposed to customers.

A category of pre-deployment testing tools has emerged to address exactly this difficulty. These tools generate adversarial or synthetic conversations, run them against an agent, and report the failures they uncover, on the premise that fixing those failures before launch will make the deployed agent more reliable. This is a reasonable premise. It is also, at present, an unproven one.

1.2 The Unvalidated Premise of Agent Testing

Every automated agent-testing system rests on a single assumption that is rarely stated and never tested: that the failures the system discovers synthetically are the failures that will actually occur in production. The field operates largely on faith. Teams run synthetic tests, fix what those tests find, and assume that production reliability improves, without ever measuring whether the distribution of synthetic failures resembles the distribution of failures that real users encounter.

The risk in this assumption runs in two directions. If synthetic testing has low recall against real failures, it misses categories of problem that production will later expose, and teams who pass their test suites are buying false confidence. If it has low precision, teams spend effort reproducing and fixing failures that would never have occurred in practice. Either way, the value of the entire activity is unknown, because the gap between synthetic and real failures has never been measured directly. Measuring it requires an uncommon combination: a working synthetic testing system on one side, and a real deployed agent whose production failures are logged independently of any testing tool on the other. That combination is the precondition this project is built around.

1.3 The Closed-Loop Gap

There is a second limitation. Even where synthetic testing is useful, it is generated blind. Tests are derived from the agent's structure and from generic assumptions about how users behave, not from any record of what has actually broken in production. No established mechanism takes a real production failure, converts it into a reproducible test, and feeds it back so that the next round of generation is sharper than the last. Closing this loop, so that production reality continuously improves pre-deployment testing, is both a research question (does grounding tests in real failures help?) and the

capability with the clearest path to becoming a defensible product. This project addresses both gaps: it measures how well synthetic testing predicts reality, and it builds and evaluates the loop that grounds testing in real failures.

1.4 Research Question

The project is organised around a single falsifiable question: *does synthetic adversarial testing predict the failures that occur in production, and does grounding the test-generation process in real production failures make it measurably better at predicting future failures than testing generated without that grounding?* This question decomposes into four sub-questions, each yielding a measurable answer.

1. **Predictive validity.** How well do the failures discovered by synthetic adversarial testing match the failures observed in real production for the same agent, measured by precision and recall over a shared failure taxonomy?
2. **Coverage gaps.** Which categories of real production failure does synthetic testing systematically miss, and what characterises them (for example long-horizon failures, real tool-integration failures not reproduced by mocks, or rare user intents)?
3. **Production feedback.** Does seeding test generation with real production failures improve predictive validity, measured by held-out recall of future production failures, compared with blind generation?
4. **Generation method.** Among generation strategies (template or persona-based, naive large-language-model prompting, and a generator-critic adversarial mode), which achieves the highest real-failure recall per unit of testing budget?

A notable feature of this design is that a negative result is as valuable as a positive one. A finding of low predictive validity would quantify, for the first time, how much trust the field should place in synthetic testing. That is a contribution the field needs, not a failure of the project.

1.5 Contributions

This project makes three contributions, which together form a build, innovate, prove arc.

Engineering contribution. The agent testing platform, an automated agent-testing system I developed, is extended with a production-feedback subsystem and presented as a working, documented artefact. The new engineering centres on a sandbox bridge that runs a real production agent behind a test endpoint, a shared failure taxonomy with a projection layer that maps failures from different sources onto it, and a measurement engine that computes predictive validity.

Research contribution. The production-feedback loop is a method that ingests real production failures, converts each into a reproducible test scenario, and re-seeds synthetic generation from them. It is introduced together with the testable hypothesis that feedback-grounded testing predicts future failures better than blind generation. This

is the novel capability of the work, and it is the one identified, on the platform's own development roadmap, as unaddressed by existing competitors.

Empirical contribution. This project produces the first measurement of the predictive validity of synthetic agent testing against real, independently-logged production failures, conducted on a live deployed commercial agent operating in Spanish across genuine customer traffic. No prior work has measured this quantity, because the required combination of systems has not previously been available to researchers.

1.6 Why This Question Is Answerable Here

The predictive-validity question has remained open less because it is uninteresting than because it is hard to put in a position to answer: it demands simultaneous access to a synthetic testing system and to a real deployed agent whose failures are recorded by processes entirely separate from that testing system. I am unusually placed to supply both. The synthetic testing system is the agent testing platform, which I built. The deployed agent is the Samsung WhatsApp customer-support agent, operated commercially by Pulpoo for Samsung, which handles real customer conversations in Spanish across a live tool catalogue.

Critically, the deployment already records its production failures through independent business processes: escalations to human agents, a two-pass escalation verifier, customer complaints, quality-assurance flags, and rule violations identified by number. These signals are generated by humans and operational systems, not by any language-model judge, which lets the study construct a ground-truth failure set that is independent of the testing tool whose accuracy it is meant to assess. The deliberate exclusion of the deployment's own language-model grader from ground truth avoids the circularity of judging one language model with another. This independence is what makes the predictive-validity question answerable rather than merely askable, and it is the foundation on which the remainder of this report builds.

1.7 Structure of This Report

The remainder of the report proceeds as follows. Chapter 2 reviews related work in agent benchmarking, adversarial testing, generator-critic generation, language-model-as-judge evaluation and its limits, and observability, and locates the gap this project addresses. Chapter 3 describes the three systems used: the agent testing platform, the agent under test, and the production evaluation infrastructure that supplies ground truth. Chapter 4 sets out the methodology, including the sandbox bridge, the frozen taxonomy, the measurement engine, the production-feedback loop, and the experimental design. The progress made to date and the remaining plan of work are reported alongside this material.

Chapter 2 Literature Review

2.1 Introduction

This chapter surveys the research and engineering literature that frames the present project, which asks two connected questions: first, whether synthetic adversarial testing of conversational and tool-using AI agents predicts the failures that those agents subsequently exhibit in live production, and second, whether a production-feedback loop that ingests real production failures, converts them into reproducible test scenarios, and re-seeds test generation can improve that predictive relationship. The review is organised around six clusters of prior work. It begins with the established agent benchmarks that define how the field currently measures agent capability, proceeds through automated and adversarial pre-deployment testing tools, examines generator and critic (adversarial multi-agent) approaches to producing test material, scrutinises the now-dominant practice of using a language model to grade another language model, and finally considers the observability and tracing platforms that teams use to watch agents in production. Each cluster is summarised analytically and then connected to the project's central concern. The chapter closes by synthesising these strands into a precisely stated research gap.

A useful organising idea throughout is the distinction, borrowed from measurement theory, between an instrument that scores performance on a fixed task and an instrument whose scores are shown to predict an externally defined outcome of interest. Raji and colleagues argue that many influential AI benchmarks suffer from construct validity problems because they are presented as general measures of capability while in fact operationalising only a narrow, decontextualised slice of behaviour, so that strong benchmark scores cannot be assumed to carry the broad meaning attributed to them [1]. The complementary concept of predictive validity, drawn from the psychometric standards jointly maintained by the American Educational Research Association, the American Psychological Association and the National Council on Measurement in Education, refers to the degree to which a test score correlates with a criterion measured at a later time rather than at the moment of testing [2]. The project's framing is that almost all existing agent evaluation work measures performance on curated tasks, whereas the question of whether such measurements predict the later criterion of real production failure has not been answered. These two reference points, construct validity and predictive validity, recur in the analysis that follows. Throughout this chapter the system developed in this project is referred to as the agent testing platform.

2.2 Agent Benchmarks and the Limits of Curated Evaluation

The first wave of agent evaluation produced standardised benchmarks that treat the large language model as an autonomous decision maker rather than as a single-turn text generator. AgentBench, introduced by Liu and colleagues and presented at the International Conference on Learning Representations in 2024, is the canonical example [3]. It assembles eight distinct interactive environments, spanning tasks from operating-system command execution to web-based interaction, and measures how well a model can reason, plan and follow instructions across multiple turns. The authors

report a wide gap between leading commercial models and open-source alternatives, attributing the most common failures to weak long-term reasoning, poor decision making and inconsistent instruction following. AgentBench is valuable because it standardises and quantifies these capabilities, but its findings are statements about average performance on a fixed, curated suite rather than predictions about any particular deployed system.

WebArena, developed by Zhou and colleagues and also presented at that conference in 2024, narrows the focus to web agents while increasing realism [4]. It provides a self-hostable environment built from functional replicas of common website categories, and scores agents on the functional correctness of long-horizon tasks that resemble those humans routinely perform online. Its authors motivate the work by noting that agents had previously been tested in simplified synthetic settings disconnected from real-world scenarios, and their environment narrows that gap considerably. It nonetheless remains a reproducible laboratory: the websites are static replicas and the tasks are authored in advance, so the benchmark measures competence on a controlled distribution rather than the behaviour an agent will display against the shifting inputs of a live user base.

The most directly relevant benchmark is the work most specific to conversational, tool-using agents. The tool-agent-user benchmark of Yao and colleagues at Sierra emulates dynamic conversations between a language-model-simulated user and an agent equipped with domain-specific tools and written policy guidelines, in retail and airline customer-service domains [5]. It compares the database state at the end of a conversation against an annotated goal state and introduces a reliability metric over repeated trials. The results are sobering: even state-of-the-art function-calling agents succeed on fewer than half of the tasks and behave inconsistently across repeated attempts, with reliability over eight trials falling below twenty-five per cent in the retail domain, underlining how far controlled competence sits from dependable real-world operation [5]. The benchmark for general AI assistants by Mialon and colleagues reinforces the point, reporting that humans answer ninety-two per cent of its questions correctly against fifteen per cent for a leading model equipped with plugins [6].

The analytical thread connecting this cluster to the project is that all of these benchmarks, however realistic, are static and curated. They establish whether an agent can perform representative tasks, but they do not and cannot tell a deploying team which failures will actually occur in their specific production environment, with their specific users, tools, policies and traffic distribution. This is the construct-validity concern of Raji and colleagues made concrete for agents: a high score on a curated suite is being asked to stand in for a much broader and context-dependent property, namely production reliability, that it was never constructed to measure [1]. The project does not seek to add another curated benchmark; it instead asks whether failures discovered before deployment correspond to failures that arrive after it, which is a question about predictive validity that benchmark leaderboards leave untouched.

2.3 Adversarial and Automated Agent Testing

A second body of work moves from measuring capability to actively provoking failure before deployment. AgentDojo, by Debenedetti and colleagues (NeurIPS 2024 Datasets and Benchmarks track), is a dynamic, extensible environment for evaluating prompt-injection attacks and defences against agents that execute tools over untrusted data, scoring outcomes against formal utility checks computed over environment state rather than relying on another model to adjudicate [7]. It demonstrates that contemporary agents both fail many tasks outright and remain vulnerable to adversarial content embedded in tool outputs. The closely related InjecAgent, by Zhan and colleagues (ACL Findings 2024), benchmarks indirect prompt injection and reports a reasoning-and-acting agent built on GPT-4 vulnerable in twenty-four per cent of cases, rising to forty-seven per cent with an added adversarial prompt [8]. AgentHarm, by Andriushchenko and colleagues with the UK AI Safety Institute (ICLR 2025), extends adversarial evaluation to malicious agentic tasks and finds that leading models are often surprisingly compliant with harmful requests even without a jailbreak [9].

A distinct automated approach is fuzzing. ToolFuzz, by Milev and colleagues in 2025, is described by its authors as the first automated method for testing agent tool usage; it combines a language model with fuzzing to generate diverse, natural user queries that cause tool runtime errors or semantically incorrect responses, targeting the under-, over- and ill-specified tool documentation that misleads tool-calling agents [10]. Evaluated on tools drawn from the LangChain framework and the commercial Composio library, it found all of the LangChain tools and most of the Composio tools to be erroneous [10], illustrating a broader category of pre-deployment tooling whose purpose is to discover failures synthetically and at scale.

The most pointed prior work for the present project is the emulated sandbox of Ruan and colleagues, presented as a spotlight at the International Conference on Learning Representations in 2024 [11]. Their framework uses a language model to emulate tool execution, allowing agents to be tested against many tools and scenarios without manually instantiating each environment, and it pairs the emulator with a model-based safety evaluator. Crucially, the authors validated their synthetic failures through human evaluation and reported that 68.8 per cent of the failures identified by the framework would be valid real-world agent failures [11]. This is the closest the literature comes to demonstrating that synthetically discovered failures are genuine, and it is an important precedent for the project. It must be read carefully, however: that figure measures the proportion of flagged failures that human annotators judged plausible and instantiable in the real world, not a measured correspondence between synthetic failures and failures actually logged in a live deployment. The distinction is the entire subject of this project.

The conclusion for this cluster is that adversarial and automated testing tools are highly effective at manufacturing or discovering failures before deployment, yet none establishes predictive validity against real production failures. They show that an agent can be made to fail, and the emulated-sandbox work shows that experts find many such failures realistic, but none demonstrates that the specific synthetic failures they surface are the same failures, in kind or in rate, that a given agent commits once serving real

users. Establishing that correspondence, rather than generating more failures, is the contribution this work aims to make.

2.4 Generator and Critic Approaches to Test Generation

The agent testing platform generates test material using a generator and critic arrangement inspired by generative adversarial networks, in which one component proposes candidate test scenarios and a second component critiques or filters them, so that the two improve adversarially. It is therefore important to locate this design within the lineage of adversarial multi-agent generation and to be precise about which prior works are genuinely relevant.

The acronym MAD-GAN is ambiguous in the literature and must be disambiguated. The relevant reference is the Multi-Agent Diverse Generative Adversarial Networks of Ghosh and colleagues, presented at the Conference on Computer Vision and Pattern Recognition in 2018 [12]. That architecture introduces multiple generators alongside a single discriminator and modifies the discriminator so that, beyond distinguishing real from generated samples, it must identify which generator produced a given sample; this pressure pushes the generators towards different high-probability modes and combats mode collapse, in which a generator produces only a narrow band of outputs. Although demonstrated on image tasks, its core idea, that an adversarial objective combined with an explicit diversity pressure yields broader coverage of the output space, is directly pertinent to test generation, where the danger is precisely that a generator settles into producing only a few repetitive kinds of scenario.

The bridge from classical adversarial generation to language-model-driven generation is provided by MALLM-GAN, the Multi-Agent Large Language Model as Generative Adversarial Network of Ling and colleagues, released in 2024 [13]. This framework synthesises tabular data by emulating the structure of a generative adversarial network using language models as the components: a generator produces synthetic data while a critic provides feedback, with the data-generation process expressed as natural-language context and the language model acting as the optimiser. The authors report improved quality of synthetic data in small-sample regimes relative to prior models. MALLM-GAN is significant for the project because it demonstrates that the generator and critic abstraction can be realised entirely with language models operating over natural-language artefacts rather than over numerical gradients, which is exactly the regime in which agent test scenarios must be produced. The broader concept it exemplifies, an adversarial or iterative generator-critic loop among language-model agents that refines generated artefacts through feedback, is the conceptual foundation for the agent testing platform's generation mode.

Connecting this cluster to the gap, the lesson from MAD-GAN and MALLM-GAN is twofold. First, an explicit critic and a diversity objective are established mechanisms for producing varied, high-coverage synthetic material, which is what a test generator needs if it is to probe an agent broadly rather than repetitively. Second, and more importantly for this project, both works optimise their generators towards an internally defined objective, namely fooling a discriminator or satisfying a critic, rather than

towards an externally validated criterion such as resembling failures that actually occur in production. A generator-critic loop can produce diverse and plausible test scenarios, but without an external signal it has no way of knowing whether those scenarios correspond to real operational risk. This observation directly motivates the project's production-feedback loop, which supplies precisely such an external signal by feeding genuine production failures back into the generation process.

2.5 LLM-as-a-Judge and the Circularity Problem

Both the adversarial testing tools and the generator-critic frameworks above frequently rely on a language model to decide whether an agent's behaviour is correct or harmful. This practice, commonly termed LLM-as-a-judge, was formalised by Zheng and colleagues in their study of judging language models with the multi-turn benchmark known as MT-Bench and with the crowdsourced Chatbot Arena, published in the Datasets and Benchmarks track of the Conference on Neural Information Processing Systems in 2023 [14]. The authors show that a strong judge model can match both controlled and crowdsourced human preferences at an agreement rate exceeding eighty per cent, which is comparable to the level of agreement between humans, and this result has made model-based grading the default for much of agent evaluation because it is scalable and inexpensive.

The same work is equally important for what it warns against. Zheng and colleagues explicitly catalogue the limitations of model judges, including position bias, in which the judge favours an answer based on its ordering; verbosity bias, in which longer answers are preferred regardless of correctness; self-enhancement bias, in which a judge favours outputs in a style resembling its own; and limited reasoning ability on tasks such as mathematics [14]. Subsequent work has confirmed and systematised these concerns. The survey of LLM-as-a-judge by Gu and colleagues, released in 2024, frames the central problem as how to build reliable model judges and surveys strategies for improving consistency and mitigating bias [15]. Dedicated bias studies, such as the quantitative investigation published as Justice or Prejudice?, define and measure roughly a dozen distinct bias types and find that current judges remain far from robust across them [16]. The recurring methodological observation across this literature is that using a language model to grade a language model introduces circularity: the instrument and the object of measurement share architectures, training data and failure modes, so the grader may be systematically blind to exactly the errors that matter most, and self-preference effects can flatter outputs that resemble the judge's own.

For the project this circularity is decisive. If synthetic adversarial testing is to be validated against real production failures, the ground truth that defines a failure cannot itself be produced by a language model, because doing so would make the validation self-referential: a model-judged synthetic failure compared against a model-judged production failure measures only the consistency of the judge, not the reality of the failure. The project therefore deliberately grounds its notion of a real production failure in independent, human-process signals, namely human escalations, customer complaints and quality-assurance flags raised by human reviewers. These signals are generated by processes outside the model under test and outside any model judge, and

they are the operational record of failures that mattered enough for a person to act. Anchoring the criterion in this way is the methodological answer to the circularity that the LLM-as-a-judge literature exposes, and it is what makes a genuine test of predictive validity possible.

2.6 Observability and Tracing: Monitoring Is Not Prediction

The final cluster concerns the platforms that teams use to watch agents once they are running. LangSmith, from the makers of the LangChain framework, captures detailed traces of agent execution, recording every step from user input to final response, including tool calls, model interactions and decision points, and surfaces latency, cost and online evaluation scores [17]. AgentOps offers comparable capabilities with an emphasis on session replays and post-hoc inspection of recorded sessions [18]. Langfuse is an open-source equivalent built natively on OpenTelemetry, organising traces of model calls and surrounding logic into traces, observations and sessions [19], and LangWatch similarly provides OpenTelemetry-native tracing and real-time monitoring across the development-to-production lifecycle [20]. Collectively these tools represent the maturing discipline sometimes called AgentOps, in which agent behaviour is instrumented and made inspectable across its lifecycle.

These platforms are genuinely useful and are not in competition with the project; rather, they supply the raw production signal that the project's feedback loop depends upon. Their analytical limitation, however, is fundamental to the gap this project addresses. Monitoring and tracing are retrospective and descriptive: they record what an agent did, expose where a failure occurred and help engineers diagnose it after the fact. The marketing language of several of these tools speaks of detecting failures, clustering them into issues and even proposing fixes, but all of these activities operate on events that have already happened. Observability tells a team what failed; it does not tell a team in advance which failures will occur, nor does it close the loop by sharpening pre-deployment testing so that those failures are caught before they reach a user. In short, monitoring is not prediction. The project treats observability output, the traces and the human-process signals attached to them, as the criterion against which synthetic testing is to be validated and as the raw material from which new test scenarios are derived, but it recognises that the act of monitoring alone does nothing to anticipate failure.

2.7 The Research Gap

Drawing the six clusters together reveals a coherent landscape with a precisely shaped hole at its centre. Curated agent benchmarks measure capability on fixed task distributions, and the construct-validity critique of Raji and colleagues shows why strong scores on such suites cannot be assumed to reflect production reliability [1, 3, 4, 5]. Adversarial and automated testing tools are highly effective at discovering failures before deployment, and the emulated-sandbox work even shows through human evaluation that a substantial majority of its synthetic failures are judged realistic, but none demonstrates that the failures they surface correspond, in kind or in frequency, to those a given agent commits in its own live deployment [7, 8, 9, 10, 11]. Generator-critic

frameworks provide powerful machinery for producing diverse synthetic material but optimise towards internally defined objectives rather than any externally validated criterion of real operational risk [12, 13]. The LLM-as-a-judge paradigm makes evaluation scalable yet introduces a circularity that disqualifies model-generated scores as independent ground truth [14, 15, 16]. And observability platforms record what happened in production with great fidelity but are inherently retrospective and do not predict what will happen [17, 18, 19, 20].

Two specific gaps emerge from this synthesis, and they are the gaps the project fills. The first concerns predictive validity. The literature contains no study that measures whether the synthetic failures discovered by adversarial testing of a conversational, tool-using agent actually match the failures that arise in that agent's real production use, where a real failure is defined by independent human-process signals rather than by a model judge. The closest precedent, the 68.8 per cent of failures judged valid by human annotators in the emulated-sandbox work, validates synthetic failures against expert plausibility judgements rather than against logged production incidents, and the broader construct-validity and benchmark-deployment literature names this gap qualitatively without closing it empirically [2, 11]. Measuring this correspondence is the project's first contribution, and it is framed explicitly as a question of predictive validity in the psychometric sense [2].

The second gap concerns the production-feedback loop. No prior work demonstrates a mechanism that takes real production failures, converts them into reproducible test scenarios and feeds them back to sharpen synthetic test generation so that future pre-deployment testing better anticipates real failure. The generator-critic literature shows that feedback can improve generated artefacts but uses only internal critics; the observability literature captures production failures but stops at description; and the adversarial testing literature generates failures without grounding generation in live operational evidence. The project's production-feedback loop joins these severed ends, using the independent human-process signals captured by observability as both the validation criterion for predictive validity and the seed material that re-targets the agent testing platform's generator-critic mode towards the failures that genuinely matter. Demonstrating that closing this loop improves the predictive relationship between synthetic and real failure is the project's second contribution. Together these two contributions move agent evaluation from measuring capability on curated tasks and from describing failures after they occur towards predicting and pre-empting the failures of a specific deployed agent, which is the practical reliability question that the existing literature, for all its strengths, leaves open.

Chapter 3 Technical Achievements and Experimentation

3.1 Overview and Scope of This Chapter

This chapter sets out the engineering work that has been completed to date, and draws a careful line between three categories of artefact: components that existed before this project began, components that have been built or substantially enhanced during this project, and components that are not yet built and that constitute the work of the next phase. The distinction matters because the contribution of an individual project is the delta over the pre-existing baseline, not the baseline itself, and because the central technical risk of the project lives squarely in the not-yet-built category. Planned work is deliberately excluded here and is treated in Chapter 4; this chapter reports only what currently exists and runs.

The honest headline is straightforward. The three core systems that the study depends upon, namely the deployed customer-support agent, the Langfuse trace substrate, and the agent testing platform, now each exist and run independently. What does not yet exist is the connective tissue between them: none of the integration layer (the sandbox bridge, the shared trace schema spanning synthetic and real runs, the frozen failure taxonomy, the projection layer, the measurement engine, and the production-feedback loop) has been built. That integration is the principal open problem heading into the next phase, and it is the precondition for every research result the project intends to produce.

3.2 Enhancements to the Agent Testing Platform

The agent testing platform existed in a baseline form before this project. The work undertaken during the project has been to extend and harden its generation and diagnosis pipeline so that it can be pointed at a real, tool-using, Spanish-language production agent rather than at toy targets. Enhancement effort concentrated on four of the platform's phases, summarised below; Phase E was deliberately left out of scope. The fuller engineering detail is recorded in Appendix A.

Phase A (agent-map analysis). Phase A produces the structured map of the agent under test that all later phases consume. It was broadened to read more source types and describe the agent more richly: priority-scored file discovery, multi-language static analysis, framework detection across twelve agent frameworks, and an LLM-driven semantic stage that infers agent purpose, per-tool semantics and risk, workflow strategy and inter-tool dependencies. Risk analysis now detects personally identifiable information and flags critical financial, data-modifying and communication actions, and the phase detects whether a conversation is in Spanish or English, which matters because the agent under test operates in Spanish.

Phase B (persona and scenario generation). Phase B generates the personas, scenarios and assembled test suite. The key enhancement was to derive persona generation directly from the agent map rather than from generic assumptions: stress-test personas are now built per tool and per multi-step tool chain, with traits derived from tool metadata. A trait-distribution analyser and a duplicate filter push for coverage rather than volume, the persona model was expanded with a mood-state

model for mood-drift simulation, and a persona-scenario affinity module pairs personas to scenarios by scored compatibility rather than at random. Spanish-language name handling, offline variant generation and per-run cost tracking were also added.

Phase C (test execution). Phase C drives multi-turn conversations against the agent and captures the resulting traces. A GAN-style adversarial mode was added, pairing a generator persona with a critic that scores conversations and applies conservative false-positive detection. The conversation simulator gained structured persona-context injection, context-aware information provision, edge-behaviour injection, and robust retry handling. On the infrastructure side, an authenticated API connector and a realistic mock connector were added, alongside a configurable chaos-injection mechanism and an AI-powered post-execution step that filters out false failures before diagnosis, so the downstream failure set is cleaner.

Phase D (failure diagnosis). Phase D clusters and explains the failures discovered in execution. Clustering was reworked into a hybrid scheme combining sentence embeddings with heuristic per-failure features, with a term-frequency fallback and a dynamically chosen cluster count. Root-cause analysis now spans twelve categories, and the phase generates minimal reproductions, proposes fixes mapped to root cause, and ranks clusters by a composite impact score. It is worth noting that this phase already implements a multi-category root-cause taxonomy; the project's shared failure taxonomy, which must span both synthetic and real failures, is a distinct artefact and has not yet been defined (see Section 3.5).

3.3 Production Observability: the Langfuse Integration

The second major piece of completed work is a full observability integration that records the deployed agent's behaviour as structured Langfuse traces. It is important to state precisely what Langfuse provides and what was built on top of it, because the contribution is the integration, not the platform. Langfuse supplies the underlying machinery out of the box: a REST client, a callback handler for LangChain-style execution, an OpenTelemetry span processor, a prompt-management API and a dataset-experiments API. None of the wiring that connects the deployed agent to that machinery existed before this project; it was written specifically for this agent.

The custom integration layer comprises several components. A trace factory wraps the Langfuse SDK with agent-specific defaults, automatically tagging the channel and conversation direction and grouping by session. A handler factory ensures the OpenTelemetry provider is initialised before the callback handler is created, then passes that handler into the agent's graph invocation so that the agent's execution auto-traces. A custom span processor stamps per-call metadata onto voice spans that would otherwise bypass the global provider. A stale-while-revalidate prompt cache was added so that prompt retrieval never blocks on the observability backend, returning a fallback immediately and refreshing in the background. An application-level scoring path records domain metrics (such as intent confidence, sentiment, escalation and resolution) on each trace after an interaction completes, and an auto-regression system listens for

prompt-version changes and triggers golden-dataset experiments with custom evaluators.

This integration is live in production. The deployed WhatsApp agent creates a trace and handler per conversation, passes the handler as a callback into its execution graph, and scores the trace with intent confidence and escalation status; production runs are recorded against Langfuse Cloud using production credentials, and the server flushes pending traces on shutdown. Production tracing of this agent did not exist before this work; it is genuinely new. As a result, the production side of the eventual comparison now has a rich, structured record of real agent behaviour, captured in a schema that the synthetic side will later be made to share.

3.4 Net State: Three Systems That Run Independently

Taken together, the completed work means that the three systems the study rests on now each exist and run. The deployed agent runs in production and is observable. The agent testing platform runs its enhanced Phase A to D pipeline and can analyse an agent, generate adversarial test suites, execute them and diagnose the resulting failures. The Langfuse substrate runs in two places: Langfuse Cloud captures production traffic, and a separate self-hosted instance has been stood up locally, intended for capturing synthetic runs so that synthetic and production data remain cleanly separated while sharing a schema.

Two points of nuance should be recorded so that nothing is overstated. First, the self-hosted Langfuse instance is stood up but currently idle: no synthetic traces have yet been written to it, because the path that would write them (the sandbox bridge and the instrumented synthetic run) does not yet exist. Second, a skeleton for connecting the systems has been started, but no integration is yet functional. The table below summarises the state of each artefact against the three categories introduced at the start of the chapter.

Existed before this project	Built or enhanced during this project	Not yet built (next phase)
The agent testing platform core (Phases A to D pipeline base).	Substantial enhancements to Phases A, B, C and D of the platform.	Sandbox bridge connecting the platform to the deployed agent.
The deployed Samsung WhatsApp support agent and its execution harness.	A complete Langfuse observability integration for the deployed agent (live in production).	Shared failure taxonomy and projection layer across synthetic and real runs.
The Langfuse platform itself	A self-hosted Langfuse instance stood up for synthetic-run capture (idle).	Measurement engine, production-feedback loop, anonymisation pipeline, evaluation harness.

Table 3.1: Status of each artefact across the three categories. The right-hand column is the subject of Chapter 4 and is the project's principal open problem.

3.5 The Central Open Problem: Integrating the Three Systems

The defining engineering challenge of the project is not the construction of any single system, since all three now run, but their integration into one closed measurement loop. Several distinct pieces of connective tissue remain, each a precondition for the research results. A sandbox bridge must wrap the deployed agent with mock tools behind an HTTP endpoint that the testing platform can drive, so synthetic tests exercise the real agent without touching live customers. The synthetic runs must be captured in the same trace schema as production, by instrumenting the sandboxed agent with the same Langfuse callback handler and directing its output to the self-hosted instance. A single frozen failure taxonomy must be defined, and a projection layer built to map both the platform's Phase D categories and the agent's independent production failure signals onto it, since comparability is impossible until both failure sources speak the same vocabulary.

Beyond these, the measurement engine that computes precision, recall, per-category breakdowns and recall-versus-budget curves, the production-feedback loop that converts a real failure into a reproducible seed scenario and re-seeds generation, and the supporting anonymisation pipeline and evaluation harness all remain to be built. Consistent with reporting only what exists, it should be stated plainly that no production conversations or failure labels have yet been extracted, no anonymisation pipeline beyond an initial connection skeleton has been written, and no pilot runs or measurements of any kind have been conducted. Prioritising the platform and the observability integration first was deliberate: there is little value in extracting and anonymising sensitive production data, or in running comparisons, before the instrument that will consume that data is known to work.

Framing this integration as the central open problem is, at the Background Report stage, an honest and defensible position. The high-risk, high-value work has been correctly identified and isolated, and the components on which it depends have been brought to a running state first. The next phase is therefore well posed: connect three working systems into a single loop, then begin the measurement the research questions require.

Chapter 4 Project Plan

This chapter sets out how the remainder of the project will be executed, what a successful outcome looks like, how the work is positioned against existing systems, and how risk is managed so that a complete and defensible deliverable is guaranteed under realistic time pressure. It closes with a mandatory statement on ethical approval, since the project uses real, anonymised production customer data.

4.1 Plan for the Remainder of the Project

The work is organised as an eight-week programme of phased delivery. The plan is deliberately front-loaded so that the single longest-lead risk, namely gaining access to real production data, securing ethical approval, and anonymising the data, is begun in week one and run in parallel with everything else. Each phase pairs a set of build tasks with the specific tests that gate them, so that no component is treated as finished until its matching test passes. Section 4.2 sets out the full schedule; the structure is that weeks one to three build a trustworthy apparatus and clean data, week four produces the first synthetic failures, week five delivers the headline predictive-validity result, and weeks six and seven add the production-feedback loop and the generation-method comparison on top.

4.2 Timeline

The schedule below presents the eight-week plan with the principal build task, the gating test and its zone, and the milestone for each week. The testing zones referenced are: Zone 0 (component tests of the project's own code), Zone 1 (sandbox fidelity against production), Zone 2 (synthetic testing of the agent), and Zone 3 (the research comparison that produces the findings).

Week	Build focus	Gating test (zone)	Milestone
1	Repo and environments; data extraction and anonymisation pipeline; self-hosted Langfuse; ethics paperwork	PII and brand detail confirmed stripped (Zone 0)	Verified-clean batch of anonymised conversations; ethics request submitted
2	Sandbox bridge over HTTP; mock tool layer with failure injection	Connector drives a full multi-turn conversation against the sandbox (Zone 0)	Platform can talk to the sandboxed agent end to end
3	Replay harness; freeze shared taxonomy; projection layer	Behavioural fidelity measured vs production (Zone 1); projection maps known failures correctly (Zone 0)	Defensible fidelity score; frozen taxonomy; both sources mappable
4	Measurement engine; run the blind arm	Engine validated on fixtures with known answers (Zone 0); first clustered synthetic set produced (Zone 2)	First end-to-end blind synthetic failure set
5	Finalise train/held-out time split	Blind synthetic vs held-out real failures (Zone 3)	Headline predictive-validity result (RQ1, RQ2); dissertation now viable

Week	Build focus	Gating test (zone)	Milestone
6	Production-feedback loop; run the feedback arm	No held-out leakage into generator (Zone 0); feedback vs blind held-out recall with significance (Zone 3)	Novel contribution measured: does feedback help, with a number (RQ3)
7	Template / naive-LLM / GAN generators; budget sweeps	Methods ranked by recall per budget (Zone 3); inter-annotator agreement reported (Zone 0/1)	All four research questions answered; results stable (RQ4)
8	Final charts and tables; assemble chapters	Key experiments re-run for reproducibility	Complete draft; reproducible results; product and future-work framing

Table 4.1: The eight-week plan, pairing each week's build focus with its gating test and milestone.

The critical-path dependency runs through the data: weeks 1 to 3 exist to make the apparatus and the data trustworthy, week 4 produces the first synthetic failures against a faithful agent, and week 5 is the first point at which the central question can be answered. Everything from week 6 onward is additive and depends on the week 5 result already being in hand.

4.3 What Success Looks Like

Success is defined concretely, as a measurable answer to each of the four research questions rather than as the construction of a working system alone.

RQ1 (predictive validity) succeeds when it yields precision and recall numbers, with per-category breakdowns, quantifying how well the failures discovered by synthetic adversarial testing match the failures observed in real production for the same agent over the shared taxonomy.

RQ2 (coverage gaps) succeeds when it yields a characterised list of the real-failure categories that synthetic testing systematically misses, together with what distinguishes them (for example long-horizon failures, real tool-integration failures not reproduced by mocks, and rare intents).

RQ3 (production feedback) succeeds when it yields a comparison of held-out recall between blind generation and feedback-grounded generation, accompanied by a statistical significance test, so that the claim that feedback improves prediction is either supported or refuted with a measured delta.

RQ4 (generation method) succeeds when it yields a ranking of generation methods (template/persona, naive LLM, GAN-style generator/critic) by real-failure recall per unit of testing budget, presented as recall-versus-budget curves.

A central point of the success criterion is that a low predictive-validity result is as valuable as a high one, for the reason set out in Section 1.4: any measured value, high or low, quantifies how much the field should trust synthetic testing. Success is therefore

the existence of a trustworthy measurement, not a particular direction of that measurement.

4.4 Comparison Against Existing Systems

The literature review positioned this work against three families of system, and its contribution is defined precisely by what none of them offers. Curated agent benchmarks such as AgentBench [3], WebArena [4], and tau-bench [5] measure how an agent performs on a fixed task set, but make no claim about whether the failures they surface correspond to those a specific deployed agent encounters with real traffic. Adversarial and fuzzing testers such as AgentDojo [7] and ToolFuzz [10], and GAN-style generation approaches [12, 13], are effective at provoking failures, but generate blind from the agent's structure with no evidence that those failures are the ones that occur in production. Observability platforms such as LangSmith [17], AgentOps [18], and Langfuse [19] record what an agent did in production but are passive monitors that surface failures after the fact and do not feed them back into sharper pre-deployment tests.

Across all three families, two capabilities are absent: a measured predictive-validity result grounded in independently logged production failures on a live deployed agent, and a closed loop that converts a real failure into a reproducible test to make the next round of testing more predictive. This project delivers both. The position is therefore not that the platform generates failures better than existing testers, but that it is the first apparatus able to measure whether synthetic failures are the right failures, and the first to ground generation in production reality.

4.5 Fallback and Risk Management

The plan is structured so that the ambitious part of the work is additive rather than load-bearing. Weeks 1 to 5 produce the complete predictive-validity study, answering RQ1 and RQ2. This study is independently complete and publishable on its own: it requires the apparatus, the frozen taxonomy, a faithful sandbox, the blind synthetic arm, and the measurement against held-out real failures, all of which are delivered by the end of week 5. Nothing load-bearing sits in the riskier later weeks.

The production-feedback loop (week 6, RQ3) and the generation-method comparison (week 7, RQ4) are layered on top. If time slips, the contingency is explicit and ordered: cut RQ4 first, then reduce RQ3 to a smaller demonstration or move it to future work. In the worst case the dissertation still stands on weeks 1 to 5; in the best case it delivers the full four-question programme including the novel feedback loop. This guarantees a complete deliverable in the worst case and an excellent one in the best case.

4.6 Ethics

This project requires Imperial College ethical approval, and approval is flagged here as required by the project specification.

The project uses real production customer conversations: Spanish-language Samsung WhatsApp support data handled by Pulpoo. Because this is real customer data, an ethics request is being submitted in week 1 of the plan, before any analysis that depends on the data is carried out, and the data-access work is sequenced so that it proceeds in step with ethical approval rather than ahead of it.

The data is handled under the following safeguards. Personally identifying information is stripped through a dedicated anonymisation pipeline combining pattern-based and named-entity recognition methods, and brand-confidential detail is scrubbed, before the data is used. The anonymisation step is itself tested in week 1 (Zone 0), with component tests that confirm PII and brand detail are actually removed, and a batch of conversations is verified clean before it is used downstream. All agent testing is performed against a sandboxed copy of the Samsung agent, never against the live production agent and never involving live customers: a faithful copy is run behind an HTTP endpoint with mock tools, so that synthetic testing exercises a stand-in rather than real customer interactions. Synthetic runs are captured on a local self-hosted Langfuse instance, keeping synthetic data cleanly separated from production data while preserving a shared schema for comparison. Finally, the production evaluation infrastructure's own automated judge scores are deliberately excluded from the ground-truth failure set; ground truth is constructed only from independent human-driven business signals such as escalations, complaints, and quality-assurance flags. This exclusion is a methodological safeguard against circularity rather than an ethical one, but it reinforces that the project's conclusions rest on independently generated, anonymised evidence.

References

[1] Raji ID, Bender EM, Paullada A, Denton E, Hanna A. AI and the Everything in the Whole Wide World Benchmark. In: Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track; 2021. Available from: arXiv:2111.15366.

[2] American Educational Research Association, American Psychological Association, National Council on Measurement in Education. Standards for Educational and Psychological Testing. Washington, DC: American Educational Research Association; 2014.

[3] Liu X, Yu H, Zhang H, Xu Y, Lei X, Lai H, et al. AgentBench: Evaluating LLMs as Agents. In: Proceedings of the International Conference on Learning Representations (ICLR); 2024. Available from: arXiv:2308.03688.

[4] Zhou S, Xu FF, Zhu H, Zhou X, Lo R, Sridhar A, et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. In: Proceedings of the International Conference on Learning Representations (ICLR); 2024. Available from: arXiv:2307.13854.

[5] Yao S, Shinn N, Razavi P, Narasimhan K. tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv preprint; 2024. Available from: arXiv:2406.12045.

[6] Mialon G, Fourrier C, Swift C, Wolf T, LeCun Y, Scialom T. GAIA: a benchmark for General AI Assistants. arXiv preprint; 2023. Available from: arXiv:2311.12983.

[7] Debenedetti E, Zhang J, Balunovic M, Beurer-Kellner L, Fischer M, Tramer F. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In: Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track; 2024. Available from: arXiv:2406.13352.

[8] Zhan Q, Liang Z, Ying Z, Kang D. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In: Findings of the Association for Computational Linguistics: ACL 2024. Association for Computational Linguistics; 2024. p. 10471-10506. Available from: arXiv:2403.02691.

[9] Andriushchenko M, Souly A, Dziemian M, Duenas D, Lin M, Wang J, et al. AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents. In: Proceedings of the International Conference on Learning Representations (ICLR); 2025. Available from: arXiv:2410.09024.

[10] Milev I, Balunovic M, Baader M, Vechev M. ToolFuzz: Automated Agent Tool Testing. arXiv preprint; 2025. Available from: arXiv:2503.04479.

[11] Ruan Y, Dong H, Wang A, Pitis S, Zhou Y, Ba J, et al. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In: Proceedings of the International Conference on Learning Representations (ICLR); 2024. Available from: arXiv:2309.15817.

[12] Ghosh A, Kulharia V, Namboodiri VP, Torr PHS, Dokania PK. Multi-Agent Diverse Generative Adversarial Networks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR); 2018. p. 8513-8521. Available from: arXiv:1704.02906.

[13] Ling Y, Tan X, Du Y. MALLM-GAN: Multi-Agent Large Language Model as Generative Adversarial Network for Synthesizing Tabular Data. arXiv preprint; 2024. Available from: arXiv:2406.10521.

[14] Zheng L, Chiang WL, Sheng Y, Zhuang S, Wu Z, Zhuang Y, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In: Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track; 2023. Available from: arXiv:2306.05685.

[15] Gu J, Jiang X, Shi Z, Tan H, Zhai X, Xu C, et al. A Survey on LLM-as-a-Judge. arXiv preprint; 2024. Available from: arXiv:2411.15594.

[16] Ye J, et al. Justice or Prejudice? Quantifying Biases in LLM-as-a-Judge. arXiv preprint; 2024. Available from: arXiv:2410.02736.

[17] LangChain. LangSmith: AI Agent and LLM Observability Platform [Internet]. 2025. Available from: https://www.langchain.com/langsmith/observability.

[18] AgentOps-AI. AgentOps Documentation: Observability and Monitoring for AI Agents [Internet]. 2025. Available from: https://docs.agentops.ai/.

[19] Langfuse. Langfuse: Open Source LLM Engineering Platform [Internet]. 2025. Available from: https://langfuse.com/.

[20] LangWatch. LangWatch: AI Agent Testing and LLM Evaluation Platform [Internet]. 2025. Available from: https://langwatch.ai/.

Appendices

This appendix records the fuller engineering detail of the platform enhancements summarised in Section 3.2. It is supporting material and is excluded from the main page count.

Appendix A Platform Enhancement Detail

A.1 Phase A: agent-map analysis

A priority-scored file-discovery stage weights agent-relevant source files (agent, tool and chain modules, workflow and prompt directories) above incidental files and detects entry points. Static analysis uses a tree-sitter-based Python parser that extracts functions with full parameter information, type annotations, docstrings, decorators, classes and imports, alongside a regular-expression parser for TypeScript and JavaScript. A framework-detection database covering twelve agent frameworks (including LangChain, LangGraph and the OpenAI and Anthropic native styles) runs with confidence scoring, together with five cumulative tool-extraction strategies that deduplicate by confidence. An LLM-driven semantic stage infers agent purpose and domain, per-tool semantics (inputs, outputs, read-only status, sensitive-data handling and per-tool risk level), workflow strategy, and inter-tool dependencies. Risk analysis adds regular-expression detection of personally identifiable information in tool parameters, descriptions and prompts, and critical-action detection that flags financial, data-modifying and communication operations with severity levels. The phase emits a versioned agent-map JSON schema and includes Spanish/English conversation-language detection.

A.2 Phase B: persona and scenario generation

Two new generators derive personas from the agent map: one creates a stress-test persona per tool (mapping a tool's risk level to adversarial versus happy-path behaviour, and a high count of required parameters to an incomplete-information edge behaviour); the other creates a persona per multi-step tool chain, with longer chains producing less patient personas. A trait-distribution analyser hints the language model about gaps before generation, and a cosine-similarity duplicate filter on numeric traits suppresses near-identical personas. The persona model was expanded from five to ten traits, with a mood-state model tracking frustration, trust and patience for mood-drift simulation and a computed archetype property. A persona-scenario affinity module scores compatibility (edge behaviour to scenario variant type, difficulty alignment, tool targeting) and selects via weighted sampling. Supporting work included nine additional template personas, Spanish-language name handling, an offline variant generator that needs no model call, hardened JSON parsing with error recovery, multi-provider model support, and per-run cost tracking across providers.

A.3 Phase C: test execution

The GAN-style adversarial mode pairs a generator persona with a critic that evaluates conversations periodically, scores quality and applies conservative false-positive detection; the critic runs as a non-blocking background task, and an offline heuristic

critic is available for model-free runs. The conversation simulator adds structured persona-context injection, context-aware information provision (supplying plausible domain values such as order or account identifiers when asked), edge-behaviour injection from the third turn onward, retry-with-backoff handling of rate limits and server errors, terminal-outcome detection, and tool-chain tracking validated against the agent map. A connector for an authenticated debug conversation API provides concurrency limiting and exponential backoff, alongside a mock connector that simulates realistic multi-tool sequences and localised confirmation messages. A configurable chaos-injection mechanism introduces faults such as mid-conversation timeouts, and an AI-powered post-execution validation step filters out false failures (for example those attributable to simulated-user incompetence or injected chaos) before diagnosis. Per-test traces, an aggregated run report and a session-resume recovery path complete the phase.

A.4 Phase D: failure diagnosis

Clustering uses a hybrid scheme: an embedding-based path (sentence embeddings with density-based clustering) is primary, with a term-frequency fallback for environments without embeddings, combining text similarity with eight heuristic features per failure (such as timeout, error type, turn count and tools called), and the cluster count is determined dynamically. Root-cause analysis spans twelve categories (for example prompt issue, tool-selection error, tool-schema mismatch, missing guardrail and hallucination), produced by a language model with a keyword-based heuristic fallback. Downstream, the phase generates minimal reproductions using a cheaper model and a shortest-first selection strategy, proposes fixes across five strategy types mapped to root cause, and ranks clusters by a composite impact score combining failure frequency, severity weighting and scope. Three run-level performance metrics (a bug-discovery rate, a redundancy rate and a severity-weighted score) were added.